PROJECT REPORT (CSN205)

A report submitted in partial fulfilment of the requirement for the course

DATABASE MANAGEMENT SYSTEMS

Part of the degree of

BACHELOR OF COMPUTER SCIENCE AND ENGINEERING

Submitted to:

Ms. Neha Singh

(Assisant Professor)

Submitted by:

Lakshay (1000025526)
Aaddeep Biswas (1000025763)
Parth Jaiswal (1000025245)
Prateek Kumar Singh (1000025863)
Section-M

SCHOOL OF COMPUTING

DIT UNIVERSITY , DEHRADUN

(State Private University through State Legislature Act No. 10 of 2013 of Uttarakhand and approved by UGC)

Mussoorie Diversion Road, Dehradun,

Uttarakhand - 248009, India.

April, 2026

Introduction

In today's time, managing student records manually is not only time-consuming but also prone to errors. In schools and colleges, there are large numbers of students, and handling their data like attendance, marks, and personal details using traditional methods becomes inefficient.

The Student Management System (SMS) is a database-based project developed to store, manage, and retrieve student-related information in an organized way. This system helps reduce paperwork and allows quick access to required data.

The main idea behind this project is to use Database Management System (DBMS) concepts such as tables, relationships, keys, and normalization to build a structured system.

Why do we need this system?

In many institutions, student data is still managed manually or using unstructured systems like Excel sheets. This leads to:

- Data redundancy
- Difficulty in searching records
- Lack of data security
- Time-consuming operations

This project aims to solve these issues by designing a proper relational database

Objectives

The main objectives of this project are:

- To maintain student records efficiently
- To manage course and enrollment details
- To track attendance and performance
- To reduce manual work and errors
- To apply DBMS concepts practically

Scope of the Project

This system can be used in:

- Schools and colleges
- Coaching institutes
- Small educational organizations

However, this project is limited to basic database operations and does not include a full frontend interface.

System Description

The system consists of multiple tables that are connected with each other using relationships.

Main components:

- Student details
- Course details
- Faculty information
- Enrollment records
- Attendance tracking
- Result management

Each table is designed carefully to avoid redundancy and ensure data consistency.

Database Design

1. Student

Stores basic details of students.

- Student_ID (Primary Key)
- Name
- Date_of_Birth
- Gender
- Phone
- Email
- Address

2. Course

Stores information about courses offered.

- Course_ID (Primary Key)
- Course_Name
- Credits

3. Faculty

Stores faculty details.

- Faculty_ID (Primary Key)
- Name
- Department

4. Enrollment

Represents relationship between student and course.

- Enrollment_ID (Primary Key)
- Student_ID (Foreign Key)
- Course_ID (Foreign Key)
- Semester

---

5. Attendance

Tracks attendance of students.

- Attendance_ID (Primary Key)
- Student_ID (Foreign Key)
- Course_ID (Foreign Key)
- Attendance_Percentage

6. Result

Stores marks and grades.

- Result_ID (Primary Key)
- Student_ID (Foreign Key)
- Course_ID (Foreign Key)
- Marks
- Grade

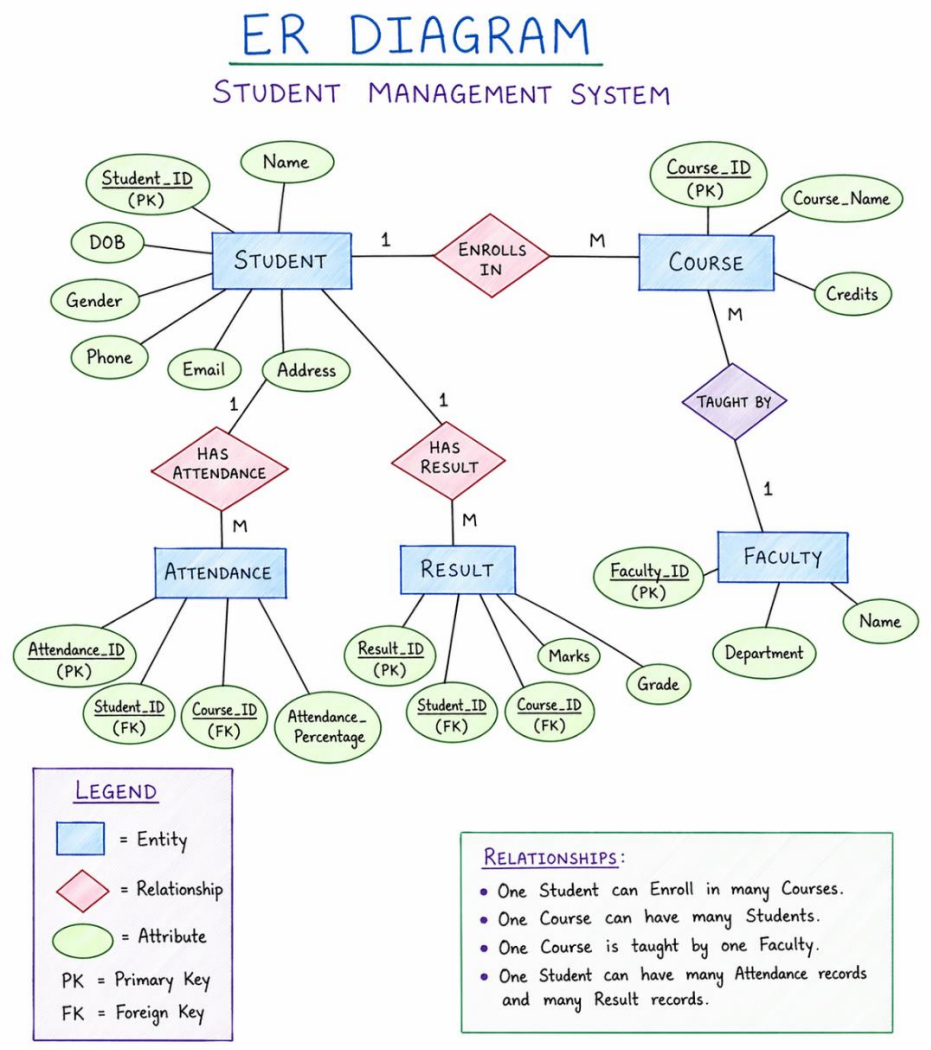
ER Diagram

The Student entity is connected to Course through the Enrollment table. This represents a many-to-many relationship, since one student can enroll in multiple courses and one course can have multiple students.

Similarly:

- Attendance and Result depend on both Student and Course
- Faculty is associated with Course

This design ensures proper relational mapping.



Relational Schema

Student(Student_ID, Name, DOB, Gender, Phone, Email, Address)

Course(Course_ID, Course_Name, Credits)

Faculty(Faculty_ID, Name, Department)

Enrollment(Enrollment_ID, Student_ID, Course_ID, Semester)

Attendance(Attendance_ID, Student_ID, Course_ID, Attendance_Percentage)

Result(Result_ID, Student_ID, Course_ID, Marks, Grade)

SQL Implementation

```
CREATE TABLE Student (
   Student_ID INT PRIMARY KEY,
   Name VARCHAR(50),
   DOB DATE,
   Gender VARCHAR(10),
   Phone VARCHAR(15),
   Email VARCHAR(50),
   Address VARCHAR(100)
);

CREATE TABLE Course (
```

```
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50),
    Credits INT
);

CREATE TABLE Faculty (
    Faculty_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50)
);

CREATE TABLE Enrollment (
    Enrollment_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    Semester INT,
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);

CREATE TABLE Attendance (
    Attendance_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    Attendance_Percentage FLOAT,
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
```

```
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);

CREATE TABLE Result (
    Result_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    Marks FLOAT,
    Grade VARCHAR(5),
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);
```

Data Insertion

```
INSERT INTO Student VALUES
(1, 'Lakshay Gaba', '2005-02-10', 'Male', '9876543210', 'lakshay@email.com', 'Dehradun'),
(2, 'Aaddeep Singh', '2004-11-05', 'Male', '9876543211', 'aaddeep@email.com', 'Delhi'),
(3, 'Parth Jain', '2005-06-18', 'Male', '9876543212', 'parth@email.com', 'Jaipur'),
(4, 'Piyush Verma', '2004-09-25', 'Male', '9876543213', 'piyush@email.com', 'Chandigarh'),
(5, 'Priya Sharma', '2005-03-12', 'Female', '9876543214', 'priya@email.com', 'Dehradun');
```

```
INSERT INTO Course VALUES
(101, 'DBMS', 4),
(102, 'Operating Systems', 4),
(103, 'Stress Management', 2),
(104, 'DAA', 4),
(105, 'Digital Electronics', 3);

INSERT INTO Faculty VALUES
(201, 'Neha Singh', 'CSE'),
(202, 'Vani Mittal', 'CSE'),
(203, 'Pappu Singh', 'Humanities'),
(204, 'Kapil Dev', 'CSE'),
(205, 'Vansh Sharma', 'ECE');

INSERT INTO Enrollment VALUES
(1, 1, 101, 2),
(2, 2, 102, 2),
(3, 3, 104, 3),
(4, 4, 105, 2),
(5, 5, 103, 1);

INSERT INTO Attendance VALUES
(1, 1, 101, 91.5),
(2, 2, 102, 85.0),
(3, 3, 104, 78.5),
(4, 4, 105, 88.0),
```

(5, 5, 103, 92.0);

INSERT INTO Result VALUES

(1, 1, 101, 89, 'A'),

(2, 2, 102, 76, 'B'),

(3, 3, 104, 82, 'A'),

(4, 4, 105, 68, 'C'),

(5, 5, 103, 91, 'A+');

Queries Used

1. Show all students

```
mysql> SELECT * FROM Student;
+------------+---------------+------------+--------+------------+-------------------+------------+
| Student_ID | Name          | DOB        | Gender | Phone      | Email             | Address    |
+------------+---------------+------------+--------+------------+-------------------+------------+
|          1 | Lakshay Gaba  | 2005-02-10 | Male   | 9876543210 | lakshay@email.com | Dehradun   |
|          2 | Aaddeep Singh | 2004-11-05 | Male   | 9876543211 | aaddeep@email.com | Delhi      |
|          3 | Parth Jain    | 2005-06-18 | Male   | 9876543212 | parth@email.com   | Jaipur     |
|          4 | Piyush Verma  | 2004-09-25 | Male   | 9876543213 | piyush@email.com  | Chandigarh |
|          5 | Priya Sharma  | 2005-03-12 | Female | 9876543214 | priya@email.com   | Dehradun   |
+------------+---------------+------------+--------+------------+-------------------+------------+
5 rows in set (0.00 sec)
```

2. Show all courses

```
mysql> SELECT * FROM Course;
+------------+---------------------+---------+
| Course_ID | Course_Name         | Credits |
+------------+---------------------+---------+
|        101 | DBMS                |       4 |
|        102 | Operating Systems   |       4 |
|        103 | Stress Management   |       2 |
|        104 | DAA                 |       4 |
|        105 | Digital Electronics |       3 |
+------------+---------------------+---------+
5 rows in set (0.00 sec)
```

3. Show all faculty

```
mysql> SELECT * FROM Faculty;
+------------+--------------+------------+
| Faculty_ID | Name         | Department |
+------------+--------------+------------+
|        201 | Neha Singh   | CSE        |
|        202 | Vani Mittal  | CSE        |
|        203 | Pappu Singh  | Humanities |
|        204 | Kapil Dev    | CSE        |
|        205 | Vansh Sharma | ECE        |
+------------+--------------+------------+
5 rows in set (0.00 sec)
```

4. Show student names with their enrolled course

SELECT s.Name, c.Course_Name

FROM Student s

JOIN Enrollment e ON s.Student_ID = e.Student_ID

JOIN Course c ON e.Course_ID = c.Course_ID;

```
mysql> SELECT s.Name, c.Course_Name
    -> FROM Student s
    -> JOIN Enrollment e ON s.Student_ID = e.Student_ID
    -> JOIN Course c ON e.Course_ID = c.Course_ID;
+---------------+---------------------+
| Name          | Course_Name         |
+---------------+---------------------+
| Lakshay Gaba  | DBMS                |
| Aaddeep Singh | Operating Systems   |
| Parth Jain    | DAA                 |
| Piyush Verma  | Digital Electronics |
| Priya Sharma  | Stress Management   |
+---------------+---------------------+
5 rows in set (0.00 sec)
```

5. Show students with attendance > 85%

SELECT s.Name, a.Attendance_Percentage

FROM Student s

JOIN Attendance a ON s.Student_ID = a.Student_ID

WHERE a.Attendance_Percentage > 85;

```
mysql> SELECT s.Name, a.Attendance_Percentage
    -> FROM Student s
    -> JOIN Attendance a ON s.Student_ID = a.Student_ID
    -> WHERE a.Attendance_Percentage > 85;
+--------------+-----------------------+
| Name         | Attendance_Percentage |
+--------------+-----------------------+
| Lakshay Gaba |                  91.5 |
| Piyush Verma |                    88 |
| Priya Sharma |                    92 |
+--------------+-----------------------+
3 rows in set (0.00 sec)
```

6. Show students who got grade 'A' or above

SELECT s.Name, r.Grade

FROM Student s

JOIN Result r ON s.Student_ID = r.Student_ID

WHERE r.Grade IN ('A', 'A+');

```
mysql> SELECT s.Name, r.Grade
    -> FROM Student s
    -> JOIN Result r ON s.Student_ID = r.Student_ID
    -> WHERE r.Grade IN ('A', 'A+');
+--------------+-------+
| Name         | Grade |
+--------------+-------+
| Lakshay Gaba | A     |
| Parth Jain   | A     |
| Priya Sharma | A+    |
+--------------+-------+
3 rows in set (0.00 sec)
```

7. Show highest marks scored

SELECT s.Name, r.Marks

FROM Student s

JOIN Result r ON s.Student_ID = r.Student_ID

WHERE r.Marks = (SELECT MAX(Marks) FROM Result);

```
mysql> SELECT s.Name, r.Marks
    -> FROM Student s
    -> JOIN Result r ON s.Student_ID = r.Student_ID
    -> WHERE r.Marks = (SELECT MAX(Marks) FROM Result);
+--------------+-------+
| Name         | Marks |
+--------------+-------+
| Priya Sharma |    91 |
+--------------+-------+
1 row in set (0.00 sec)
```

8. Show student + course + marks (full report)

SELECT s.Name, c.Course_Name, r.Marks, r.Grade

FROM Student s

JOIN Result r ON s.Student_ID = r.Student_ID

JOIN Course c ON r.Course_ID = c.Course_ID;

```
mysql> SELECT s.Name, c.Course_Name, r.Marks, r.Grade
    -> FROM Student s
    -> JOIN Result r ON s.Student_ID = r.Student_ID
    -> JOIN Course c ON r.Course_ID = c.Course_ID;
+---------------+---------------------+-------+-------+
| Name          | Course_Name         | Marks | Grade |
+---------------+---------------------+-------+-------+
| Lakshay Gaba  | DBMS                |    89 | A     |
| Aaddeep Singh | Operating Systems   |    76 | B     |
| Parth Jain    | DAA                 |    82 | A     |
| Piyush Verma  | Digital Electronics |    68 | C     |
| Priya Sharma  | Stress Management   |    91 | A+    |
+---------------+---------------------+-------+-------+
5 rows in set (0.00 sec)
```

9. Show faculty teaching each course

```
mysql> SELECT c.Course_Name, f.Name AS Faculty_Name
    -> FROM Course c
    -> JOIN Faculty f ON c.Course_ID = f.Faculty_ID - 100;
+---------------------+--------------+
| Course_Name         | Faculty_Name |
+---------------------+--------------+
| DBMS                | Neha Singh   |
| Operating Systems   | Vani Mittal  |
| Stress Management   | Pappu Singh  |
| DAA                 | Kapil Dev    |
| Digital Electronics | Vansh Sharma |
+---------------------+--------------+
5 rows in set (0.00 sec)
```

Advantages

- Easy data management
- Reduces redundancy
- Faster access to information
- Better organization
- Scalable for future improvements

Limitations

- No user interface (only backend)
- Requires basic SQL knowledge
- Not suitable for very large institutions without upgrades

Conclusion

The Student Management System successfully demonstrates how DBMS concepts can be applied in real-life scenarios. It simplifies the process of storing and managing student data while maintaining accuracy and consistency.

This project helped in understanding database design, relationships, and SQL queries in a practical way.